AISC5000 Foundation of Machine Learning and AI
Lecture 6: Training Neural Networks with Gradient Descent

Bangti Jin (b.jin@cuhk.edu.hk)

October 22, 2026

Outline

1 Introduction

2 Error Surfaces

3 Gradient Descent Optimization

4 Initialization

5 Convergence

6 Advanced Optimizers

7 Normalization

8 Practical Training Issues

Big picture

- A neural network is a flexible function approximator:

$$f(\mathbf{x}; \mathbf{w})$$

  where $\mathbf{w}$ denotes all trainable weights and biases.
- Training means choosing $\mathbf{w}$ to minimize a loss/error function:

$$\min_{\mathbf{w}} E(\mathbf{w}).$$

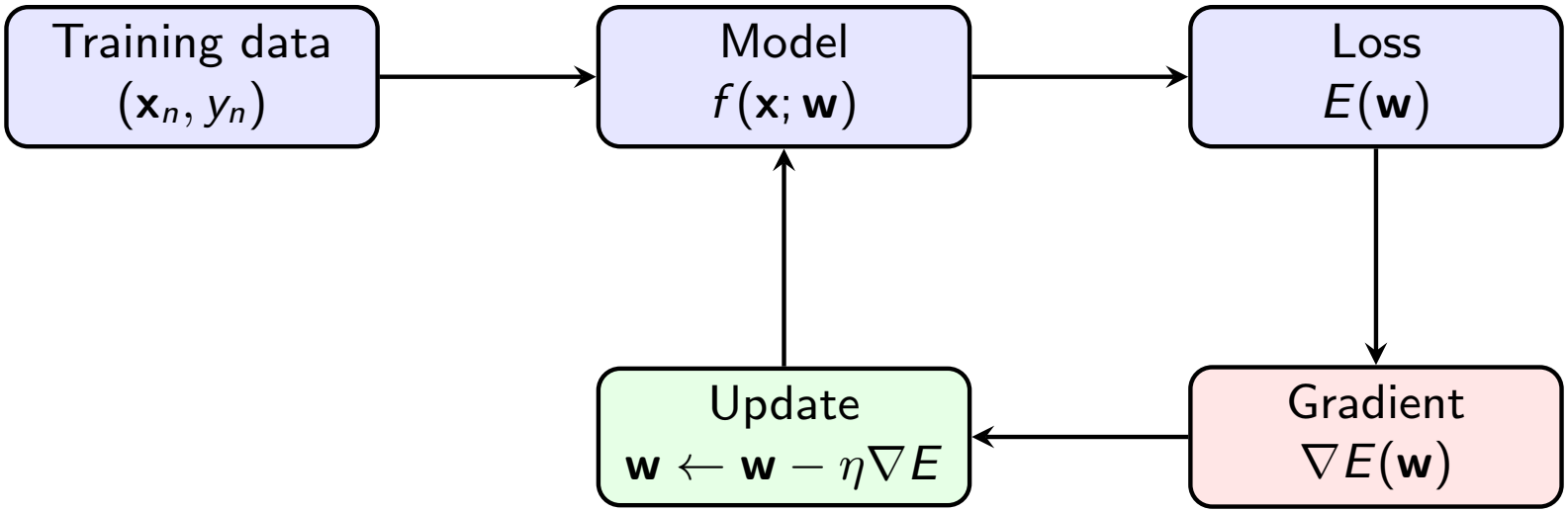
- For regression, a common loss is mean squared error:

$$E(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^{N} (y_n - f(\mathbf{x}_n; \mathbf{w}))^2 .$$

- For classification, a common loss is cross-entropy.
- Modern networks can have millions or billions of parameters.
- Main challenge:

  Find good parameters efficiently and make the model generalize.

Training pipeline



Key idea

Training is an iterative loop:

predict $\rightarrow$ measure error $\rightarrow$ compute gradient $\rightarrow$ update parameters.

Motivating example: Suppose we fit a line

$$\hat{y} = wx$$

to one data point $(x, y) = (2, 4)$.
Use squared loss:

$$E(w) = \frac{1}{2}(wx - y)^2 = \frac{1}{2}(2w - 4)^2 = 2(w - 2)^2.$$

The minimum is clearly at $w^* = 2$.
But in real neural networks, $E(\mathbf{w})$ is much more complicated. Gradient descent gives a general recipe.

Example: Consider $E(w) = 2(w-2)^2$. Then

$$E'(w) = 4(w-2).$$

Gradient descent update:

$$w^{(\tau+1)} = w^{(\tau)} - \eta E'(w^{(\tau)}).$$

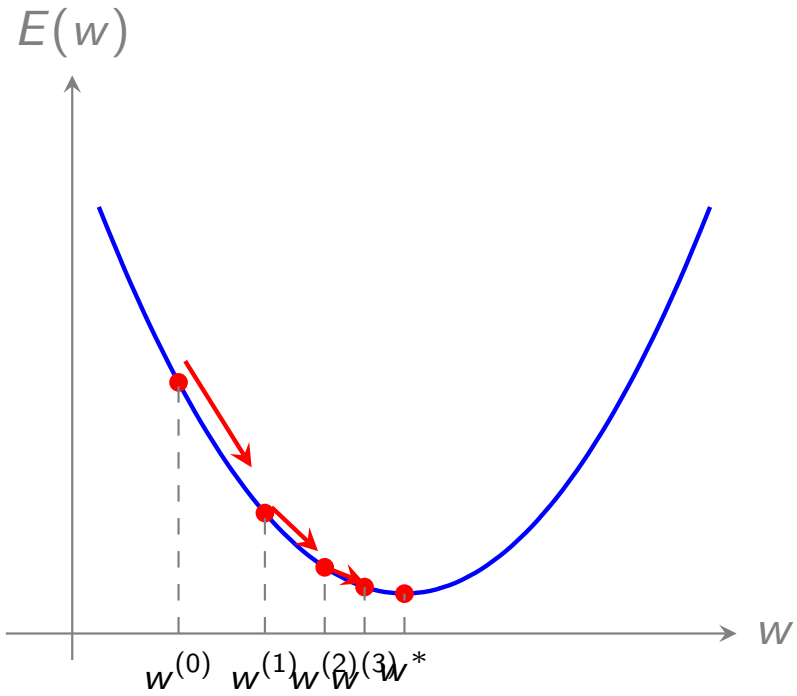
Take $\eta = 0.1$ and $w^{(0)} = 0$.

$$w^{(1)} = 0 - 0.1 \cdot 4(0-2) = 0.8,$$
$$w^{(2)} = 0.8 - 0.1 \cdot 4(0.8-2) = 1.28,$$
$$w^{(3)} = 1.28 - 0.1 \cdot 4(1.28-2) = 1.568.$$

Observation

The iterates move closer to $w^* = 2$.

1D gradient descent



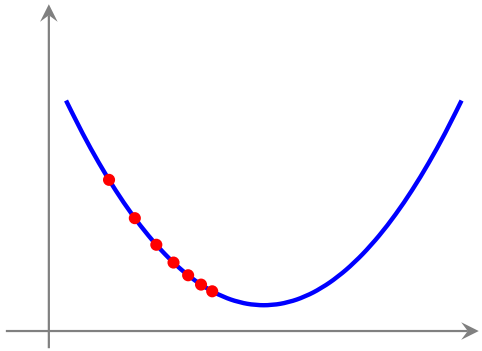
$$w \leftarrow w - \eta E'(w).$$

Move opposite the derivative because the derivative points uphill.

What can go wrong?

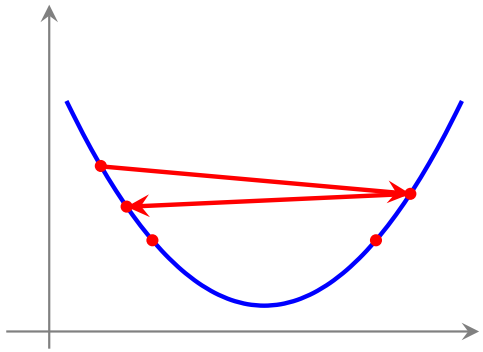
If η is too small

- Convergence is stable.
- But progress is very slow.
- Training wastes computation.



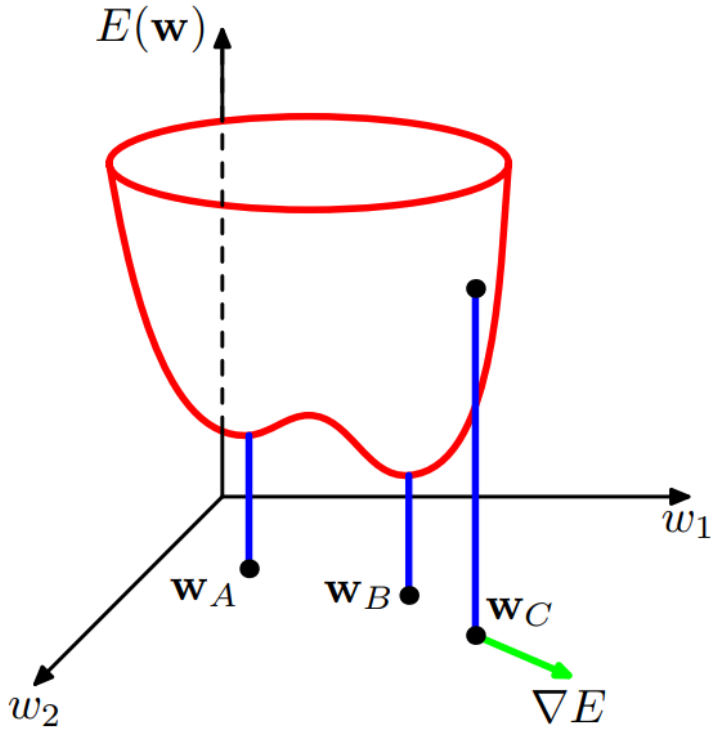
If η is too large

- Updates may overshoot the minimum.
- Loss may oscillate or diverge.
- Training becomes unstable.



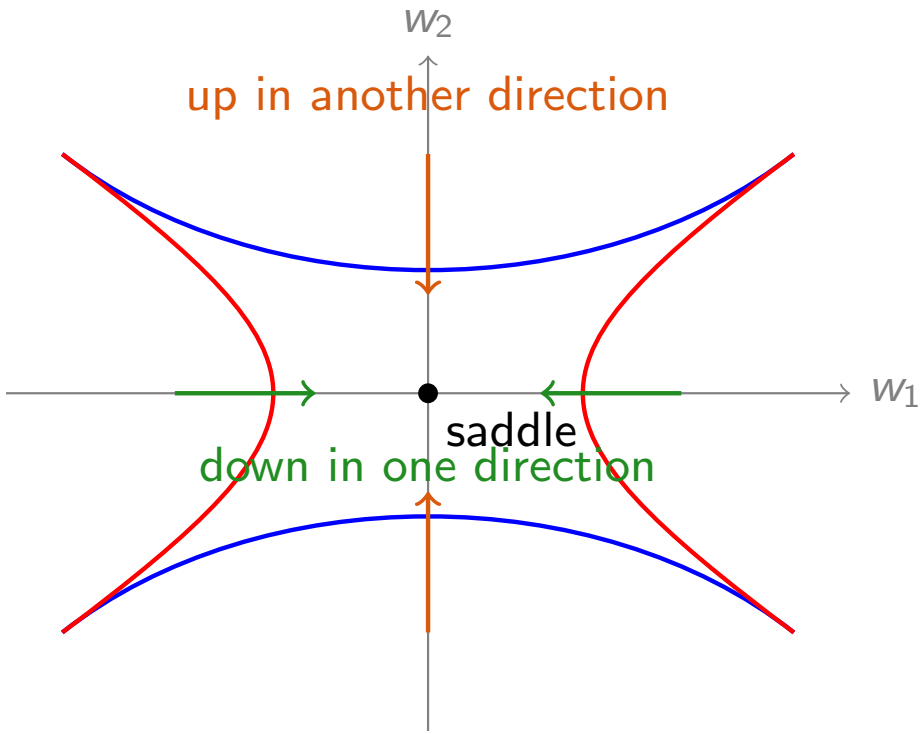
Error surfaces

- $E(\mathbf{w})$ defines a surface over weight space.
- Goal: find a low point of this surface.
- gradient $\nabla E(\mathbf{w})$ points uphill.
- negative gradient $-\nabla E(\mathbf{w})$ points downhill.
- stationary points satisfy $\nabla E(\mathbf{w}) = 0$
- Stationary points can be:
  - local minima,
  - global minima,
  - local maxima,
  - saddle points.



In high-dimensional neural networks, saddle points are often more important than local maxima.

saddle points



Example: $E(w_1, w_2) = w_1^2 - w_2^2$.
At $(0, 0)$: $\nabla E = 0$, but it is neither a minimum nor a maximum.

Local quadratic approximation

To understand optimization dynamics, approximate the error surface locally by Taylor expansion around $\widehat{\mathbf{w}}$:

$$E(\mathbf{w}) \simeq E(\widehat{\mathbf{w}}) + (\mathbf{w} - \widehat{\mathbf{w}})^T \mathbf{b} + \frac{1}{2}(\mathbf{w} - \widehat{\mathbf{w}})^T \mathbf{H}(\mathbf{w} - \widehat{\mathbf{w}}).$$

- $\mathbf{b} = \nabla E|_{\widehat{\mathbf{w}}}$ is the gradient vector.
- $\mathbf{H}$ is the Hessian matrix:

$$H_{ij} = \frac{\partial^2 E}{\partial w_i \partial w_j}.$$

At a minimum $\mathbf{w}^*$, $\mathbf{b} = 0$, so

$$E(\mathbf{w}) \simeq E(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^T \mathbf{H}(\mathbf{w} - \mathbf{w}^*).$$

The Hessian

- The Hessian is symmetric under standard smoothness assumptions.
- Therefore it has real eigenvalues and orthogonal eigenvectors:

$$\mathbf{H}\mathbf{u}_i = \lambda_i \mathbf{u}_i, \qquad \mathbf{u}_i^T \mathbf{u}_j = \delta_{ij}.$$

- Express displacement from a minimum:

$$\mathbf{w} - \mathbf{w}^* = \sum_i \alpha_i \mathbf{u}_i.$$

- Then

$$E(\mathbf{w}) = E(\mathbf{w}^*) + \frac{1}{2} \sum_i \lambda_i \alpha_i^2.$$

Interpretation

Along eigenvector direction $\mathbf{u}_i$, the curvature is λ_i.

Classifying stationary points

At a stationary point:

$$\nabla E(\mathbf{w}^*) = 0.$$

Use the eigenvalues of the Hessian H:

Hessian eigenvalues	**Type of point**
All $\lambda_i > 0$	local minimum
All $\lambda_i < 0$	local maximum
Some positive, some negative	saddle point
Some $\lambda_i = 0$	inconclusive / flat direction

Positive definite Hessian

$$H \succ 0 \quad \Longleftrightarrow \quad \mathbf{v}^T H \mathbf{v} > 0 \quad \text{for all} \quad \mathbf{v} \neq 0.$$

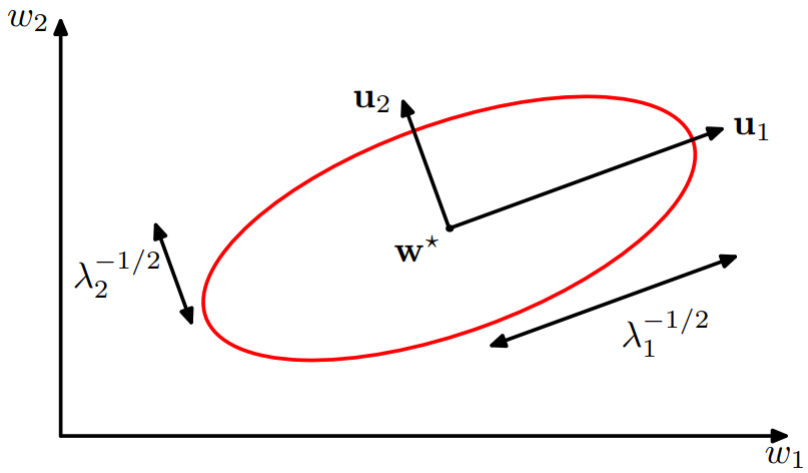
If H is symmetric, this is equivalent to $\lambda_i > 0$ for all i.

Contours of constant error

For a quadratic error near a minimum:

$$E(\mathbf{w}) = E(\mathbf{w}^*) + \frac{1}{2}\sum_i \lambda_i \alpha_i^2.$$

The contours $E(\mathbf{w}) = \text{constant}$ are ellipses.



- Large λ_i: steep direction.
- Small λ_i: flat direction.
- Large condition number causes slow zig-zagging.

Why gradient information is crucial

- Direct optimization without gradients requires many function evaluations.
- With W parameters, the error surface lives in W-dimensional space.
- A single scalar loss value $E(\mathbf{w})$ tells us only whether one point is good or bad.
- The gradient

$$\nabla E(\mathbf{w}) = \left(\frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_W} \right)^T$$

  tells us how the loss changes in every coordinate direction.
- Backpropagation computes this gradient efficiently.

Main point

Gradients make high-dimensional optimization feasible.

Generic gradient-type method

A general iterative optimization method has the form:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \Delta\mathbf{w}^{(\tau-1)}.$$

- τ: iteration step.
- $\mathbf{w}^{(0)}$: initialization.
- Different methods choose different updates $\Delta\mathbf{w}$.

Method	**Update idea**
Gradient descent	move opposite full gradient
SGD	use one sample gradient
Mini-batch SGD	use a small batch gradient
Momentum	smooth updates using velocity
Adam	momentum + adaptive coordinate scaling

Batch gradient descent

For a loss function

$$E(\mathbf{w}) = \frac{1}{N}\sum_{n=1}^{N} E_n(\mathbf{w}),$$

batch gradient descent uses the full gradient:

$$\nabla E(\mathbf{w}) = \frac{1}{N}\sum_{n=1}^{N} \nabla E_n(\mathbf{w}).$$

Update:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E(\mathbf{w}^{(\tau-1)}).$$

- $\eta > 0$: learning rate.
- Pros: stable, deterministic direction.
- Cons: expensive for large datasets
- Each update requires processing all N data points

Stochastic gradient descent

SGD uses one randomly selected training example at a time:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E_n(\mathbf{w}^{(\tau-1)}).$$

- n is chosen randomly, usually after shuffling data.
- One full pass through the data is called an **epoch**.
- Much cheaper per update than batch gradient descent.
- But the gradient estimate is noisy, can have high variance
- can escape local minima due to noise, handle redundant data efficiently

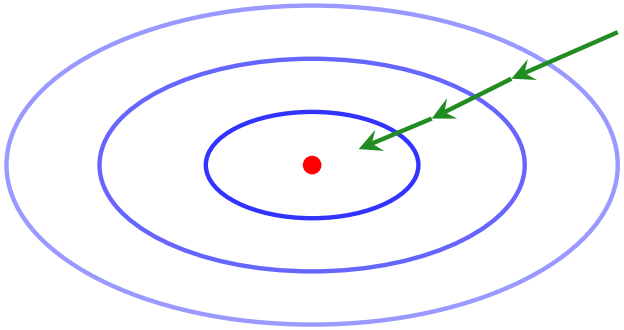
Important property

If n is sampled uniformly, then

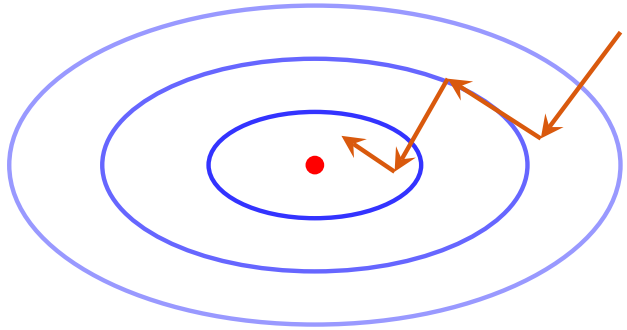
$$\mathbb{E}_n[\nabla E_n(\mathbf{w})] = \nabla E(\mathbf{w})$$

up to the convention of sum versus average loss.

batch GD vs. SGD



Batch GD



SGD

- Batch GD follows a smoother path.
- SGD follows a noisy path but updates more frequently.

Algorithm: stochastic gradient descent

Stochastic Gradient Descent

1. Initialize $\mathbf{w}$ randomly.
2. Repeat for each epoch:
    1. Shuffle the training data.
    2. For each data point n:
        - Compute $\nabla E_n(\mathbf{w})$.
        - Update
        $$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_n(\mathbf{w}).$$
3. Stop when the validation performance no longer improves.

In practice, pure one-sample SGD is less common than mini-batch SGD.

Mini-batch gradient descent

Mini-batch gradient descent is a compromise between batch GD and SGD. Choose a mini-batch $\mathcal{B}$ of size B. Then

$$\nabla E_{\mathcal{B}}(\mathbf{w}) = \frac{1}{B} \sum_{n \in \mathcal{B}} \nabla E_n(\mathbf{w})$$

Update:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E_{\mathcal{B}}(\mathbf{w}^{(\tau-1)}).$$

- Typical batch sizes: $32, 64, 128, 256$.
- Reduces variance compared to one-sample SGD.
- Much cheaper than full-batch GD.
- Efficient on GPUs due to vectorized matrix operations.
- **best practice**: randomly shuffle data before each epoch to avoid correlations (i.e., constituent data points should be randomly chosen)

Batch, SGD, and mini-batch comparison

Method	Data per update	Noise	Cost per update
Batch GD	all N samples	low	high
SGD	one sample	high	low
Mini-batch GD	B samples	medium	medium

Practical rule

Most deep learning training uses mini-batch stochastic optimization.

$$B = 1 \quad \longrightarrow \quad \text{SGD}, \qquad B = N \quad \longrightarrow \quad \text{Batch GD}.$$

Algorithm: mini-batch gradient descent

Mini-Batch Gradient Descent

1. Initialize $\mathbf{w}$ randomly.
2. Choose batch size B.
3. Repeat for each epoch:
   1. Shuffle the training data.
   2. Split data into mini-batches.
   3. For each mini-batch $\mathcal{B}$:
      - Compute
      $$\nabla E_{\mathcal{B}}(\mathbf{w}) = \frac{1}{B} \sum_{n \in \mathcal{B}} \nabla E_n(\mathbf{w}).$$
      - Update
      $$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_{\mathcal{B}}(\mathbf{w}).$$
4. Until convergence.

Parameter initialization

- Initialization affects convergence speed and final performance.
- If all hidden units start with the same weights, they compute the same function.
- Random initialization breaks symmetry.
- Common choices:

$$w_{ij} \sim \mathcal{N}(0, \epsilon^2) \quad \text{or} \quad w_{ij} \sim U[-\epsilon, \epsilon].$$

- ϵ should not be too small or too large.

Bad initialization

- Too small: signals and gradients may vanish.
- Too large: activations and gradients may explode.

Why not initialize all weights to zero?

Consider two hidden units with the same input:

$$z_1 = \sigma(\mathbf{w}_1^T \mathbf{x}), \qquad z_2 = \sigma(\mathbf{w}_2^T \mathbf{x}).$$

If $\mathbf{w}_1^{(0)} = \mathbf{w}_2^{(0)}$, then the two units compute the same output. They also receive the same gradient update, so

$$\mathbf{w}_1^{(\tau)} = \mathbf{w}_2^{(\tau)}$$

for all iterations τ.

Consequence

The network wastes capacity: multiple hidden units learn the same feature.

Random initialization breaks this symmetry.

He initialization for ReLU networks

For a layer with M inputs:

$$a_i^{(l)} = \sum_{j=1}^{M} w_{ij} z_j^{(l-1)}, \qquad z_i^{(l)} = \mathrm{ReLU}(a_i^{(l)}).$$

Assume:

$$w_{ij} \sim \mathcal{N}(0, \epsilon^2), \qquad \mathrm{Var}(z_j^{(l-1)}) = \lambda^2.$$

For ReLU, roughly half the activations are zero, so

$$\mathrm{Var}(z_i^{(l)}) \approx \frac{M}{2} \epsilon^2 \lambda^2.$$

To keep variance stable across layers, choose

$$\frac{M}{2} \epsilon^2 \approx 1.$$

Thus

$$\epsilon = \sqrt{\frac{2}{M}}.$$

This is the idea behind He initialization.

Transfer learning and pre-training

- Instead of random initialization, use weights learned from another task.
- Example:
  - pre-train on ImageNet,
  - fine-tune on a smaller medical-image dataset.
- This is called **transfer learning**.
- Benefits:
  - faster convergence,
  - better generalization,
  - less labeled data needed.
- Unsupervised pre-training, e.g. autoencoders, can also provide a useful starting point.

Convergence of gradient descent near a minimum

Consider the quadratic approximation near $\mathbf{w}^*$:

$$E(\mathbf{w}) = E(\mathbf{w}^*) + \frac{1}{2}\sum_i \lambda_i \alpha_i^2.$$

The gradient in the eigenvector basis is

$$\nabla E = \sum_i \lambda_i \alpha_i \mathbf{u}_i.$$

Gradient descent gives

$$\alpha_i^{\text{new}} = (1 - \eta\lambda_i)\alpha_i^{\text{old}}.$$

After T steps:

$$\alpha_i^{(T)} = (1 - \eta\lambda_i)^T \alpha_i^{(0)}.$$

Convergence requires

$$|1 - \eta\lambda_i| < 1 \quad \text{for all } i.$$

Therefore:

$$0 < \eta < \frac{2}{\lambda_{\max}}.$$

Derivation of the componentwise update

Let

$$\mathbf{w} - \mathbf{w}^* = \sum_i \alpha_i \mathbf{u}_i.$$

For the quadratic model:

$$\nabla E = H(\mathbf{w} - \mathbf{w}^*) = H \sum_i \alpha_i \mathbf{u}_i.$$

Since

$$H\mathbf{u}_i = \lambda_i \mathbf{u}_i,$$

we get

$$\nabla E = \sum_i \alpha_i \lambda_i \mathbf{u}_i.$$

Gradient descent update:

$$\Delta \mathbf{w} = -\eta \nabla E = -\eta \sum_i \alpha_i \lambda_i \mathbf{u}_i.$$

Therefore:

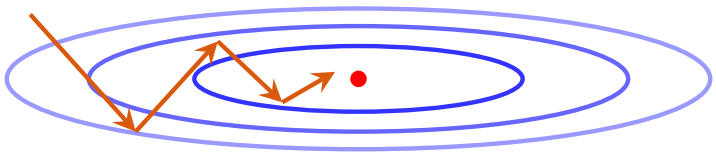
$$\Delta \alpha_i = -\eta \lambda_i \alpha_i,$$

Condition number and ill-conditioning

The **condition number** of the Hessian is

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$$

- If κ is small, the contours are close to circular.
- If κ is large, the contours form a narrow valley.
- Step size is limited by $\lambda_{\max}$.
- Progress along the direction with $\lambda_{\min}$ is slow.
- Plain gradient descent can zig-zag and be inefficient for ill-conditioned problem.



Momentum method

Momentum adds a velocity term:

$$\mathbf{v}^{(\tau)} = \mu \mathbf{v}^{(\tau-1)} - \eta \nabla E(\mathbf{w}^{(\tau-1)}),$$

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \mathbf{v}^{(\tau)}.$$

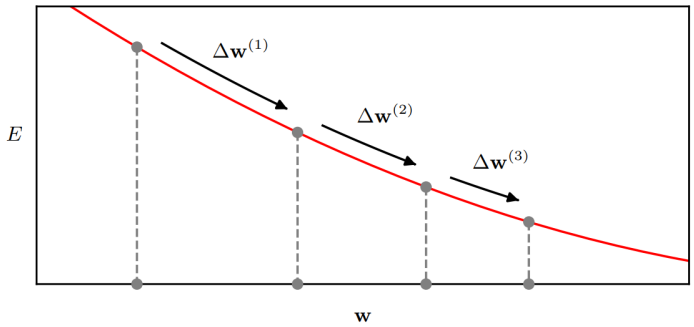
- $\mu \in [0, 1]$: momentum coefficient, with typical value: $\mu = 0.9$.
- Momentum accumulates a running average of past gradients.
- It accelerates motion in consistent directions.
- It reduces oscillations in directions where gradients change sign.

If the gradient is approximately constant

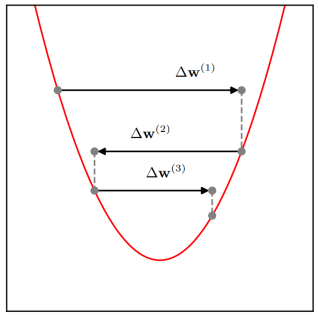
$$\Delta w \approx -\eta \nabla E(1 + \mu + \mu^2 + \cdots) = -\frac{\eta}{1-\mu} \nabla E.$$

momentum reduces zig-zagging

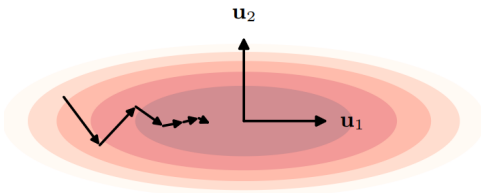
impact of the momentum on training



low curvature



high curvature



reduced oscillation

Momentum smooths the trajectory through a narrow valley.

Algorithm: SGD with momentum

SGD with Momentum

1. Initialize $\mathbf{w}$, set $\mathbf{v} = 0$.
2. Choose learning rate η and momentum μ.
3. Repeat:
   1. Compute mini-batch gradient:
   $$\mathbf{g} = \nabla E_{\text{batch}}(\mathbf{w}).$$
   2. Update velocity:
   $$\mathbf{v} \leftarrow \mu\mathbf{v} - \eta\mathbf{g}.$$
   3. Update weights:
   $$\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}.$$
4. Until convergence.

Nesterov momentum

Standard momentum computes the gradient at the current position.
Nesterov momentum computes the gradient at a look-ahead point:

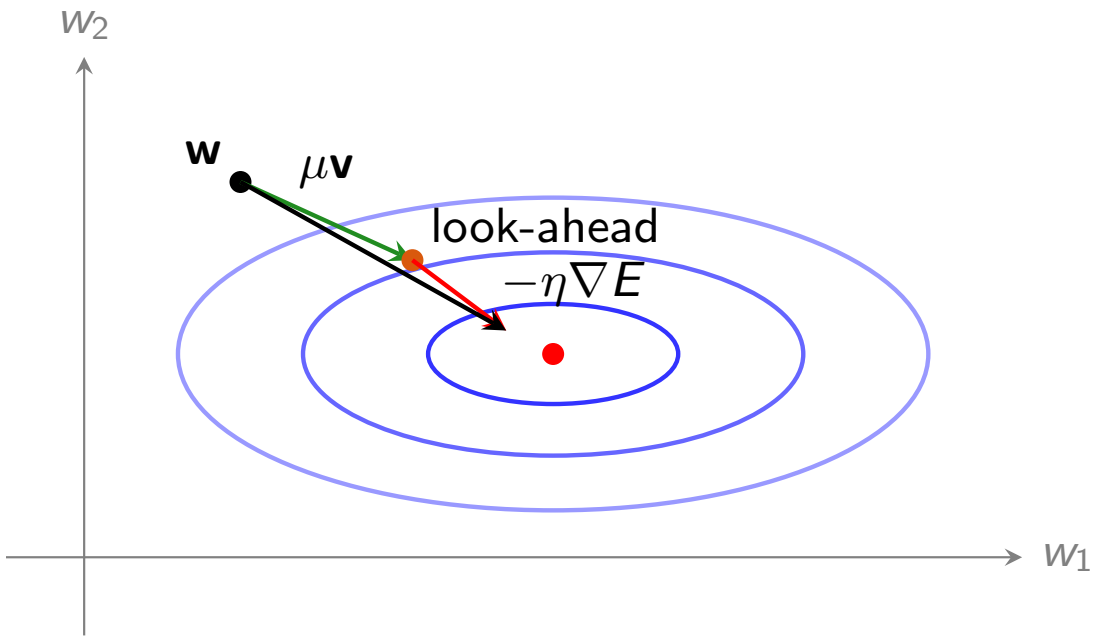
$$\mathbf{w}^{(\tau-1)} + \mu\mathbf{v}^{(\tau-1)}.$$

Update:

$$\mathbf{v}^{(\tau)} = \mu\mathbf{v}^{(\tau-1)} - \eta\nabla E\left(\mathbf{w}^{(\tau-1)} + \mu\mathbf{v}^{(\tau-1)}\right),$$
$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \mathbf{v}^{(\tau)}.$$

- This provides a more accurate correction and can improve convergence, especially for batch GD.
- For SGD, the benet is sometimes less pronounced.

Nesterov look-ahead



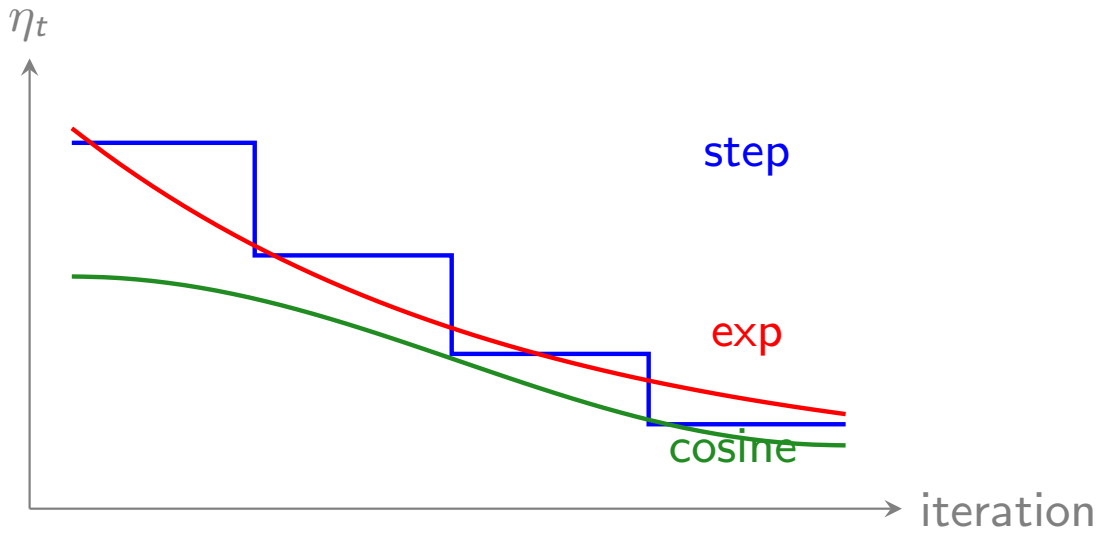
Learning rate schedules

A fixed learning rate is often suboptimal. Common schedules:

- Step decay:
$$\eta_t = \eta_0 \gamma^{\lfloor t/s \rfloor}.$$
- Exponential decay:
$$\eta_t = \eta_0 \exp(-kt).$$
- Power decay:
$$\eta_t = \eta_0 (1 + t/s)^{-c}.$$
- Linear decay:
$$\eta_t = \eta_0 (1 - t/T), \qquad t < T.$$
- Cosine decay:
$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T}\right).$$

Choosing the schedule is often done by monitoring the validation error.

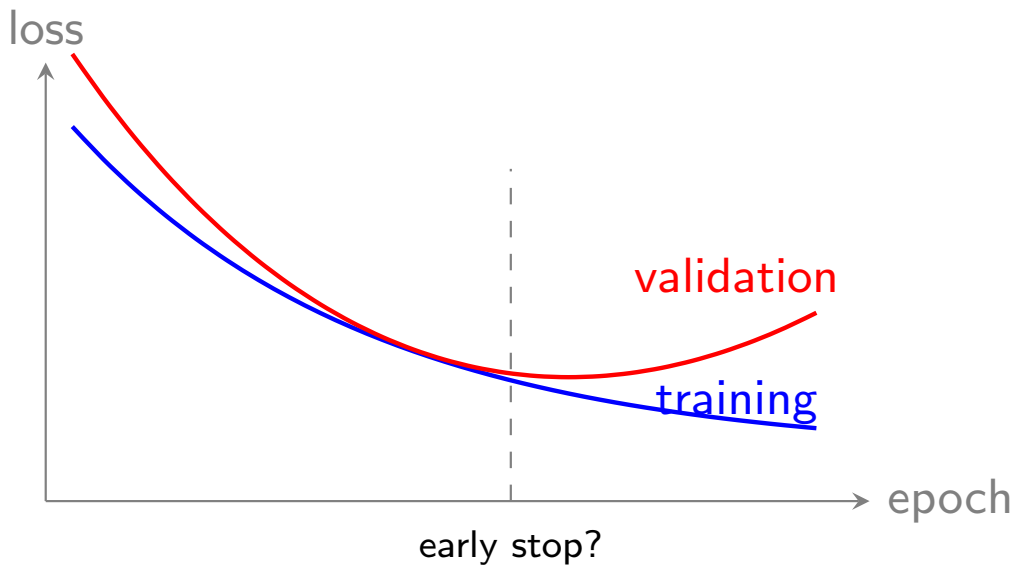
learning rate schedules



Start large for rapid progress, then decrease for fine tuning.

Monitoring the learning curve

It is essential to plot training and validation error.



- If loss oscillates wildly, η may be too high.
- If loss decreases very slowly, η may be too low.
- If validation loss increases while training loss decreases, overfitting may be occurring.

Adaptive learning rates

- The best learning rate can vary by parameter.
- Some parameters receive large gradients.
- Other parameters receive small or sparse gradients.
- Adaptive methods adjust the learning rate for each parameter individually.

General form

$$w_i \leftarrow w_i - \frac{\eta}{\text{scale}_i} g_i.$$

The scale is estimated from past gradients.

Important caveat

Adaptive methods often assume coordinate-wise scaling is useful. This is most accurate when curvature directions are roughly aligned with coordinate axes.

AdaGrad

AdaGrad accumulates squared gradients:

$$r_i^{(\tau)} = r_i^{(\tau-1)} + \left(g_i^{(\tau)}\right)^2.$$

Update:

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)}} + \delta} g_i^{(\tau)}.$$

- δ is a small constant, e.g. 10^{-8}.
- Large past gradients $\Rightarrow$ smaller future steps.
- Small or sparse gradients $\Rightarrow$ relatively larger steps.

Pros and cons

- Pro: good for sparse gradient, e.g. NLP features.
- Con: r_i only grows, so learning rates can become too small.

Example: Suppose a single parameter has gradients

$$g^{(1)} = 2, \qquad g^{(2)} = 1.$$

Let

$$r^{(0)} = 0, \qquad \eta = 0.1, \qquad \delta = 0.$$

Step 1:

$$r^{(1)} = 0 + 2^2 = 4.$$

Effective learning rate: $\frac{\eta}{\sqrt{r^{(1)}}} = \frac{0.1}{2} = 0.05.$

Update size:

$$0.05 \times 2 = 0.1.$$

Step 2:

$$r^{(2)} = 4 + 1^2 = 5.$$

Effective learning rate: $\frac{0.1}{\sqrt{5}} \approx 0.0447.$

Observation

The effective learning rate decreases over time.

RMSProp

RMSProp (root mean square propagation) fixes AdaGrad's decay problem using an exponentially weighted moving average:

$$r_i^{(\tau)} = \beta r_i^{(\tau-1)} + (1-\beta)\left(g_i^{(\tau)}\right)^2.$$

Update:

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)}} + \delta} g_i^{(\tau)}.$$

- β is often around 0.9.
- Recent gradients matter more than old gradients.
- The effective learning rate does not monotonically vanish.
- Useful for non-stationary objectives.

Adam: adaptive moment estimation

Adam combines momentum and RMSProp. Kingma & Ba 2014 First moment estimate:

$$s_i^{(\tau)} = \beta_1 s_i^{(\tau-1)} + (1-\beta_1) g_i^{(\tau)}.$$

Second moment estimate:

$$r_i^{(\tau)} = \beta_2 r_i^{(\tau-1)} + (1-\beta_2)(g_i^{(\tau)})^2.$$

Bias correction:

$$\widehat{s}_i^{(\tau)} = \frac{s_i^{(\tau)}}{1-\beta_1^\tau}, \qquad \widehat{r}_i^{(\tau)} = \frac{r_i^{(\tau)}}{1-\beta_2^\tau}.$$

Update:

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \eta \frac{\widehat{s}_i^{(\tau)}}{\sqrt{\widehat{r}_i^{(\tau)}} + \delta}.$$

momentum for each parameter separately: exponentially weighted moving averages for gradient and squared gradient

- Typical values: $\beta_1 = 0.9$, $\beta_2 = 0.99$, $\eta = 0.001$
- Adam is the most popular optimizer in deep learning.

Why does Adam need bias correction?

Adam initializes

$$s^{(0)} = 0, \qquad r^{(0)} = 0.$$

Therefore, early moving averages are biased toward zero. For example, if the gradient is constant g, then

$$s^{(1)} = (1 - \beta_1)g.$$

This is much smaller than g when β_1 is close to 1. Bias correction divides by

$$1 - \beta_1^{\tau}$$

so that, for constant gradients,

$$\hat{s}^{(\tau)} \approx g.$$

Similarly:

$$\hat{r}^{(\tau)} \approx g^2.$$

Algorithm: Adam optimizer

Adam Optimizer

1. Initialize $\mathbf{w}$, $\mathbf{s} = 0$, $\mathbf{r} = 0$.
2. Choose η, β_1, β_2, δ.
3. For each iteration τ:
    1. Sample mini-batch and compute gradient $\mathbf{g}$.
    2. $\mathbf{s} \leftarrow \beta_1 \mathbf{s} + (1 - \beta_1)\mathbf{g}$.
    3. $\mathbf{r} \leftarrow \beta_2 \mathbf{r} + (1 - \beta_2)\mathbf{g} \odot \mathbf{g}$.
    4. $\widehat{\mathbf{s}} \leftarrow \mathbf{s}/(1 - \beta_1^\tau)$.
    5. $\widehat{\mathbf{r}} \leftarrow \mathbf{r}/(1 - \beta_2^\tau)$.
    6. Update:
$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\widehat{\mathbf{s}}}{\sqrt{\widehat{\mathbf{r}}} + \delta}.$$
4. Until convergence.

Comparison of optimizers

Method	Momentum	Adaptive LR	Bias corr.	Common use
SGD	No	No	No	baseline
SGD + mom.	Yes	No	No	CNNs, strong baseline
AdaGrad	No	Yes, sum	No	sparse data
RMSProp	No	Yes, EMA	No	RNNs, non-stationary
Adam	Yes	Yes, EMA	Yes	common default
AdamW	Yes	Yes, EMA	Yes	transformers, weight de

Practical comment

Adam often works well out of the box. SGD with momentum can sometimes give better final generalization, especially in vision models.

Practical considerations for optimizers

- **Learning rate:** usually the most important hyperparameter.
- **Batch size:** affects gradient noise and hardware efficiency.
- **Momentum:** often improves SGD.
- **Weight decay:** discourages large weights and helps regularization.
- **Gradient clipping:** prevents exploding gradients.
- **Warmup:** gradually increase learning rate at the beginning.
- **Validation monitoring:** avoid overfitting and tune schedules.

Debugging rule

If training fails, check the learning rate first.

Why normalize?

Input features may have very different scales.
Example:

$$x_1 = \text{age in years}, \qquad x_2 = \text{income in dollars}.$$

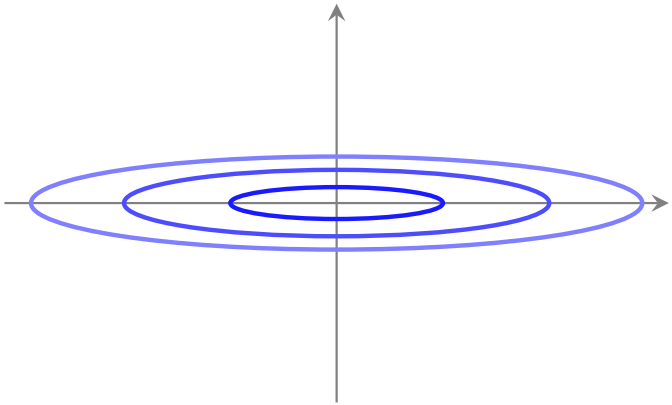
Without normalization, the loss surface may be ill-conditioned.
Normalization helps by:

1. speeding up convergence,
2. improving numerical stability,
3. reducing sensitivity to initialization,
4. avoiding saturation of nonlinearities such as sigmoid and tanh.

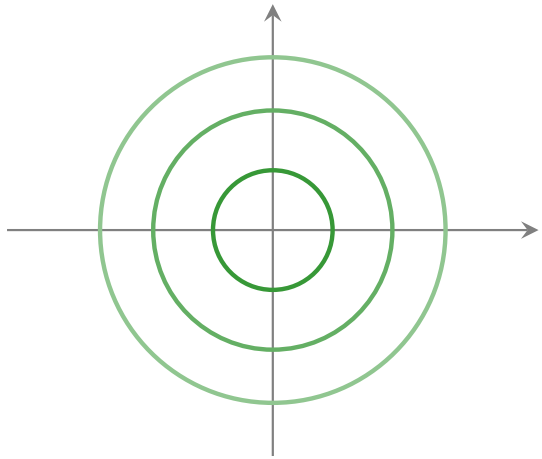
Three important kinds:

- data normalization,
- batch normalization,
- layer normalization.

normalization improves conditioning



before normalization



after normalization

Normalization can make the optimization landscape easier for gradient descent.

Data normalization

Compute mean and variance per feature over the training set:

$$\mu_i = \frac{1}{N}\sum_{n=1}^{N} x_{ni},$$

$$\sigma_i^2 = \frac{1}{N}\sum_{n=1}^{N}(x_{ni} - \mu_i)^2.$$

Transform:

$$\widetilde{x}_{ni} = \frac{x_{ni} - \mu_i}{\sigma_i}.$$

Then each feature has approximately zero mean and unit variance.

Important

Use the training-set μ_i and σ_i for validation and test data.

Data normalization: Suppose one feature has training values:

$$2, \quad 4, \quad 6.$$

Mean:

$$\mu = \frac{2+4+6}{3} = 4.$$

Variance:

$$\sigma^2 = \frac{(2-4)^2 + (4-4)^2 + (6-4)^2}{3} = \frac{8}{3}.$$

Standard deviation:

$$\sigma = \sqrt{\frac{8}{3}}.$$

Normalized values:

$$\frac{2-4}{\sqrt{8/3}}, \qquad \frac{4-4}{\sqrt{8/3}}, \qquad \frac{6-4}{\sqrt{8/3}}.$$

Approximately:

$$-1.225, \quad 0, \quad 1.225.$$

Batch normalization

Batch normalization normalizes activations within a mini-batch.
For mini-batch size K, unit i:

$$\mu_i = \frac{1}{K}\sum_{n=1}^{K} a_{ni},$$

$$\sigma_i^2 = \frac{1}{K}\sum_{n=1}^{K}(a_{ni} - \mu_i)^2,$$

$$\widehat{a}_{ni} = \frac{a_{ni} - \mu_i}{\sqrt{\sigma_i^2 + \delta}}.$$

Then apply learnable scale and shift:

$$\widetilde{a}_{ni} = \gamma_i \widehat{a}_{ni} + \beta_i.$$

Why γ_i, β_i? Normalization may remove useful scale and offset. The network can learn them back if needed.

Worked example: batch normalization

For one hidden unit, suppose the mini-batch pre-activations are:

$$a_1 = 1, \quad a_2 = 2, \quad a_3 = 5.$$

Mean and variance:

$$\mu = \frac{1+2+5}{3} = \frac{8}{3}.$$

$$\sigma^2 = \frac{1}{3}\left[\left(1-\frac{8}{3}\right)^2 + \left(2-\frac{8}{3}\right)^2 + \left(5-\frac{8}{3}\right)^2\right] = \frac{26}{9}.$$

Thus

$$\widehat{a}_n = \frac{a_n - \mu}{\sqrt{\sigma^2 + \delta}}.$$

If $\gamma = 2$, $\beta = 1$, then

$$\widetilde{a}_n = 2\widehat{a}_n + 1.$$

Takeaway

BN standardizes each unit's mini-batch activations, then lets the model rescale and shift.

Benefits of batch normalization

- Allows higher learning rates.
- Reduces sensitivity to initialization.
- Helps stabilize signal and gradient magnitudes.
- Adds noise through mini-batch statistics, which can have a regularizing effect.
- Often speeds up training significantly.

More detailed intuition

In deep networks, each layer receives inputs whose distribution changes as earlier layers update. BN keeps intermediate activations in a controlled range, making optimization easier.

Modern view

BN's success is not only due to reducing internal covariate shift. It can also smooth the optimization landscape and improve gradient behavior.

Batch normalization: training vs. inference

During training: μ_i, σ_i^2 are computed from the current mini-batch.
During inference:

- We may have one test example at a time.
- Mini-batch statistics would be noisy or unavailable.
- Instead, use moving averages collected during training.

Moving averages:

$$\overline{\mu}_i^{(\tau)} = \alpha \overline{\mu}_i^{(\tau-1)} + (1-\alpha)\mu_i^{(\tau)},$$
$$\overline{\sigma}_i^{2,(\tau)} = \alpha \overline{\sigma}_i^{2,(\tau-1)} + (1-\alpha)\sigma_i^{2,(\tau)}.$$

At test time:

$$\widehat{a}_i = \frac{a_i - \overline{\mu}_i}{\sqrt{\overline{\sigma}_i^2 + \delta}}.$$

Layer normalization

Layer normalization normalizes across hidden units for each data point. For a layer with M units and data point n:

$$\mu_n = \frac{1}{M}\sum_{i=1}^{M} a_{ni},$$

$$\sigma_n^2 = \frac{1}{M}\sum_{i=1}^{M}(a_{ni} - \mu_n)^2,$$

$$\widehat{a}_{ni} = \frac{a_{ni} - \mu_n}{\sqrt{\sigma_n^2 + \delta}}.$$

Then apply learnable scale and shift

$$\widetilde{a}_{ni} = \gamma_i \widehat{a}_{ni} + \beta_i.$$

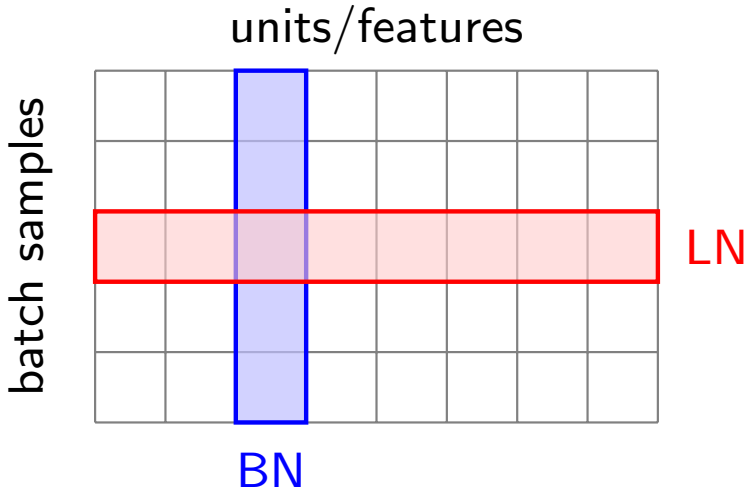
- Independent of batch size.
- Same behavior during training and inference.
- Standard in transformers.

BatchNorm vs. LayerNorm

Consider an activation matrix:

$$A \in \mathbb{R}^{K \times M}$$

where rows are samples and columns are units.



BN: normalize down columns; LN: normalize across rows

BN vs. LN

Batch Normalization

- Normalizes across batch dimension for each unit.
- Depends on batch size.
- Uses moving averages at test time.
- Works well for CNNs and feedforward networks.

Layer Normalization

- Normalizes across feature dimension for each sample.
- Independent of batch size.
- Same formula at training and test time.
- Works well for RNNs, transformers, and small batches.

Other normalization techniques

- **Instance normalization:** normalize per sample per channel. Often used in style transfer.
- **Group normalization:** divide channels into groups and normalize within each group. Useful when batch size is small.
- **Weight normalization:** reparameterize weights as

$$\mathbf{w} = g \frac{\mathbf{v}}{\|\mathbf{v}\|}.$$

- The best choice depends on:
  - architecture,
  - batch size,
  - task,
  - hardware constraints.

Vanishing and exploding gradients

For a deep network, gradients involve products of many Jacobian matrices:

$$\frac{\partial E}{\partial \mathbf{h}^{(l)}} = \left(\prod_{k=l+1}^{L} J_k^T \right) \frac{\partial E}{\partial \mathbf{h}^{(L)}}.$$

If many Jacobian singular values are less than 1, gradients vanish.
If many are greater than 1, gradients explode.

Remedies

- ReLU or variants.
- Careful initialization, e.g. He initialization.
- Normalization.
- Residual connections.
- Gradient clipping for exploding gradients.

Gradient clipping

Exploding gradients are common in deep networks, especially recurrent neural networks. Gradient clipping caps the gradient before applying the update.

Value clipping

Clip each gradient coordinate:

$$g_i \leftarrow \max(-c, \min(g_i, c)).$$

Norm clipping

If

$$\|\mathbf{g}\| > c,$$

then replace

$$\mathbf{g} \leftarrow c\frac{\mathbf{g}}{\|\mathbf{g}\|}.$$

This prevents very large parameter updates.

norm clipping: Suppose

$$\mathbf{g} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

Then

$$\|\mathbf{g}\|_2 = 5.$$

Let the clipping threshold be

$$c = 2.$$

Since $5 > 2$, norm clipping gives

$$\mathbf{g}_{\text{clipped}} = 2\frac{\mathbf{g}}{\|\mathbf{g}\|} = 2\frac{1}{5}\begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1.2 \\ 1.6 \end{bmatrix}.$$

The direction is unchanged, but the length is reduced to 2.

Residual connections help gradients flow

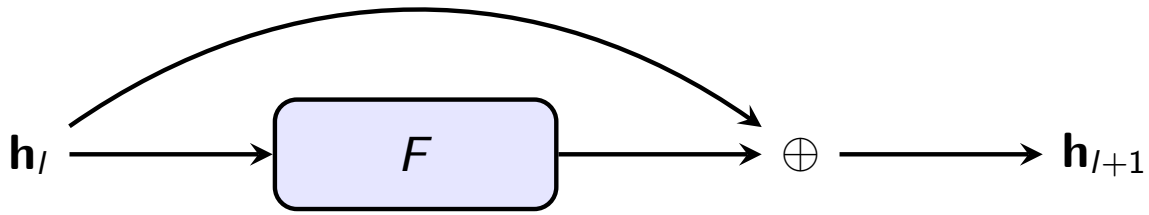
A residual block computes

$$\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l; \mathbf{w}_l).$$

The derivative includes an identity term:

$$\frac{\partial \mathbf{h}_{l+1}}{\partial \mathbf{h}_l} = I + \frac{\partial F}{\partial \mathbf{h}_l}.$$

Why helpful? If $\frac{\partial F}{\partial \mathbf{h}_l}$ is small, the identity path allows gradient information to pass backward.



Hyperparameter tuning

Key hyperparameters: learning rate, batch size, momentum, weight decay, learning rate schedule, initialization, normalization method. Common search strategies:

- grid search,
- random search,
- Bayesian optimization,
- learning rate finder.

Practical advice

Random search is often more efficient than grid search when only a few hyperparameters matter strongly.

Debugging training: symptoms and fixes

Symptom	Possible fix
Training loss is NaN	reduce learning rate; check data; use gradient clipping
Loss does not decrease	increase learning rate; check gradients; check labels
Loss oscillates wildly	reduce learning rate; use momentum carefully
Training accuracy high, validation low	regularization; data augmentation; early stopping
Both training and validation poor	larger model; better features; train longer
Gradients are zero	check activation saturation; initialization; learning rate
Gradients explode	gradient clipping; normalization; smaller learning rate

Practical advice for training

- Start with a simple model and a small dataset to debug.
- Overfit a tiny subset first. If the model cannot overfit 10 examples, something is wrong.
- Monitor training and validation metrics.
- Tune learning rate before other hyperparameters.
- Use normalization when appropriate.
- Use gradient clipping for unstable recurrent or very deep models.
- Use early stopping to avoid overfitting.
- Adam is a good default optimizer.
- SGD with momentum can sometimes give better final test accuracy.

Summary

- Training neural networks means minimizing an error function $E(\mathbf{w})$.
- Gradient descent moves in the negative gradient direction.
- Mini-batch methods balance computational efficiency and gradient noise.
- Learning rate is the most important hyperparameter.
- Momentum accelerates learning and reduces oscillation.
- Adam combines momentum and adaptive learning rates.
- Initialization and normalization strongly affect stability.
- Gradient clipping helps with exploding gradients.
- Always monitor training and validation curves.
- Goal: good generalization, not just low training loss.

References


Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.


Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010*, pages 177–186.


Kingma, D. P. and Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.


Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*.


Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.


He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *ICCV*.